AMD AI DEVMASTER HACKATHON 2026

AgentFlow AI

Offline Multi-Agent AI Assistant

for Private Document Intelligence



Project Specification Document

Technical design, product rationale, and implementation plan

Prepared for Track 2 submission

6 August 2026

Private by architecture. Intelligent by design.

Contents

1 Executive Summary 3

Executive Summary 3

1.1 Product Positioning . . . 3

2 Problem and Opportunity 4

Problem and Opportunity 4

2.1 Target Users . . . 4

2.2 Opportunity . . . 4

3 Product Experience 5

Product Experience 5

3.1 Core Interaction Loop . . . 5

4 Functional Specification 6

Functional Specification 6

4.1 Feature Catalogue . . . 6

4.2 Non-Functional Requirements . . . 6

4.3 Scope Boundary . . . 7

5 Multi-Agent Design 8

Multi-Agent Design 8

5.1 Coordination Logic . . . 8

6 System Architecture 9

System Architecture 9

6.1 Data Boundaries . . . 9

7 Data and Retrieval Pipeline 10

Data and Retrieval Pipeline 10

7.1 Retrieval Quality Controls 10
7.2 Memory Model 10

8 Workflow and State Management 11

Workflow and State Management 11

8.1 Failure and Recovery 11

9 Technology Stack 12

Technology Stack 12

9.1 Why This Stack 12

10 AMD Optimization Strategy 13

AMD Optimization Strategy 13

10.1 Benchmarking Plan 13

11 Security, Privacy, and Governance 14

Security, Privacy, and Governance 14

11.1 Responsible AI Position 14

12 Delivery Roadmap and Future Scope 15

Delivery Roadmap and Future Scope 15

12.1 Future Scope 15

13 Advantages and Evaluation 16

Advantages and Evaluation 16

13.1 Hackathon Demonstration Script 16
13.2 Success Measures 16

14 Conclusion 17

Conclusion 17

References 18

1 Executive Summary

AgentFlow AI is an offline, multi-agent document intelligence platform designed for organizations that cannot place sensitive information in third-party cloud services. It turns a collection of private PDFs into an interactive, locally searchable knowledge environment. Users upload documents through a modern React dashboard, ask questions in natural language, request summaries, and continue conversations without losing context. The entire inference and retrieval path remains on the user-controlled machine.

The system combines Retrieval-Augmented Generation, persistent conversation memory, and a coordinated set of purpose-built agents. A Planner Agent decomposes the request; a Research Agent locates evidence; a Memory Agent recalls relevant prior interactions; a Tool Agent performs structured operations; and a Reviewer Agent checks the result before it reaches the user. This separation is deliberate. It makes the system easier to understand, test, extend, and govern than a single undifferentiated chatbot.

AgentFlow AI is particularly relevant to enterprise, government, healthcare, legal, education, and research environments. These sectors routinely work with documents that are valuable precisely because they are confidential. The product proposition is straightforward: provide the practical utility of modern AI while removing the default requirement to export private documents to the cloud.

100%	5	0	1
local processing	coordinated agents	cloud API dependency	unified workspace

Investment thesis

AgentFlow AI addresses a growing trust gap in enterprise AI. Its modular architecture creates a credible path from hackathon prototype to an on-premise knowledge product, while local deployment provides a clear differentiation against hosted document assistants.

1.1 Product Positioning

The product is not merely a chat interface placed over a vector database. Its value lies in orchestration. Each agent has a narrow responsibility, observable inputs and outputs, and an explicit place in the execution plan. That design supports reliable iteration: retrieval can be improved without rewriting memory, the reviewer can be strengthened without changing the interface, and AMD acceleration can be introduced without redesigning the user experience.

2 Problem and Opportunity

Organizations are accumulating documents faster than people can read them. Contracts, operating procedures, research papers, audit records, clinical notes, and internal policies are often distributed across folders with inconsistent naming and weak search. Traditional keyword search finds strings, but it does not reliably answer questions such as "What changed between these two policy versions?" or "Which section supports this recommendation?"

Cloud AI services improve accessibility, yet they introduce a difficult trade-off. Sensitive files may leave the organization, retention policies may be unclear, and network availability becomes part of the user experience. For regulated teams, the technical ability to use an external service is not the same as permission to do so. The gap between AI capability and acceptable data handling remains a practical barrier to adoption.

Current friction	Operational consequence	AgentFlow response
Documents are fragmented across local repositories	Time is lost locating authoritative passages	ChromaDB creates a semantic, local index
Single-shot chat loses working context	Repeated explanations reduce confidence and speed	SQLite preserves conversation memory
Cloud APIs are unsuitable for restricted material	Teams delay or abandon AI adoption	Ollama keeps inference on the deployment machine
One general model performs every task	Errors are difficult to diagnose and improve	Specialized agents create observable stages

2.1 Target Users

- **Enterprise analysts** reviewing policies, operating manuals, and internal knowledge.
- **Government and defence teams** working with restricted material in disconnected or controlled environments.
- **Healthcare and legal professionals** needing concise summaries without transferring sensitive records.
- **Students and researchers** exploring large collections of papers, reports, and technical references.

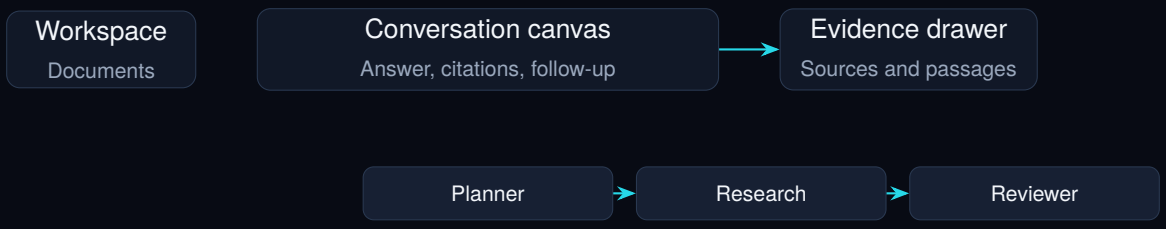
2.2 Opportunity

The strongest initial opportunity is not a universal consumer assistant. It is a trustworthy, deployable intelligence layer for teams that already own their documents and need a faster way to reason over them. A local-first design also creates a practical foundation for enterprise deployment: administrators can control models, storage, network access, and retention policies in one place.

3 Product Experience

The experience begins with a calm, focused dashboard rather than a crowded configuration console. A left navigation rail exposes workspaces, indexed documents, and conversation history. The central panel presents the active dialogue, while a contextual evidence drawer shows retrieved passages and document references. A compact agent activity strip communicates what the system is doing without forcing the user to understand the orchestration details.

The interface uses a dark navy foundation with cyan action states, purple intelligence indicators, and orange emphasis for decisions or warnings. Glass-like panels separate information without visually fragmenting the workspace. Subtle gradients and restrained glow effects communicate technical sophistication, but the visual hierarchy remains practical: the answer is primary, evidence is inspectable, and system status is never hidden.



3.1 Core Interaction Loop

1. The user uploads one or more PDFs or selects an existing knowledge workspace.
2. The system displays indexing progress and confirms when content is searchable.
3. A request is entered in natural language, such as a summary, comparison, or question.
4. The dashboard presents the final response alongside supporting evidence and agent status.
5. The user can continue the conversation, refine the scope, or save the result for later review.

Design principle

Trust should be visible. AgentFlow AI exposes provenance, processing state, and memory boundaries so that users can understand not only what the assistant answered, but why the answer was produced.

4 Functional Specification

4.1 Feature Catalogue

Capability	User value	Acceptance signal
Offline AI assistant	Work without internet access or cloud credentials	Inference completes with network disabled
PDF ingestion	Turn documents into searchable knowledge	Upload status and indexed document count are visible
Semantic retrieval	Find meaning, not only exact words	Relevant passages appear for paraphrased questions
Document summarization	Reduce reading time for long files	Summary contains scope, themes, and supporting references
Question answering	Obtain direct, contextual answers	Responses cite retrieved document context
Multi-agent collaboration	Divide complex work into manageable stages	Agent trace shows plan, research, memory, tools, and review
Conversation history	Resume work without repeating context	Prior sessions can be reopened from SQLite
Modern dashboard	Make complex AI behavior approachable	Key tasks are reachable within one workspace

4.2 Non-Functional Requirements

- **Privacy:** no document content, embeddings, prompts, or responses leave the local deployment by default.
- **Reliability:** failures in one tool or retrieval step must produce a clear recoverable state rather than a silent answer.
- **Explainability:** responses should expose source passages and an execution summary where appropriate.
- **Maintainability:** agents, storage adapters, and model providers must be replaceable through defined interfaces.
- **Performance:** the application should provide progressive feedback during indexing and inference instead of appearing frozen.
- **Security:** local storage paths, document identifiers, and session data should be isolated per workspace.

4.3 Scope Boundary

The first release focuses on PDF text understanding. It does not promise perfect interpretation of handwritten pages, complex scanned tables, or images embedded in documents. Those cases are intentionally treated as future extensions so the initial product can deliver a dependable text-first experience.

5 Multi-Agent Design

AgentFlow AI uses a supervisor-style pipeline in which each agent contributes a bounded capability. The Planner Agent converts an ambiguous request into a sequence of actions. The Research Agent retrieves relevant context from ChromaDB and ranks it for usefulness. The Memory Agent retrieves prior conversation facts from SQLite while respecting the active workspace. The Tool Agent handles structured actions such as document metadata lookup or summarization preparation. Finally, the Reviewer Agent checks coherence, grounding, and obvious contradictions before the answer is returned.

This pattern is valuable because complexity becomes inspectable. A failure can be classified as planning, retrieval, memory, tool execution, or review rather than being attributed vaguely to "the model." It also encourages smaller prompts, clearer test cases, and more predictable future upgrades.

Agent	Primary responsibility	Output
Planner Agent	Interpret intent and create an execution plan	Ordered task graph and constraints
Research Agent	Retrieve and organize relevant document passages	Evidence bundle with source identifiers
Memory Agent	Recall useful prior turns and persistent facts	Scoped conversation context
Tool Agent	Execute structured local operations	Tool result or explicit failure state
Reviewer Agent	Check grounding, completeness, and response quality	Approved response or revision request

5.1 Coordination Logic

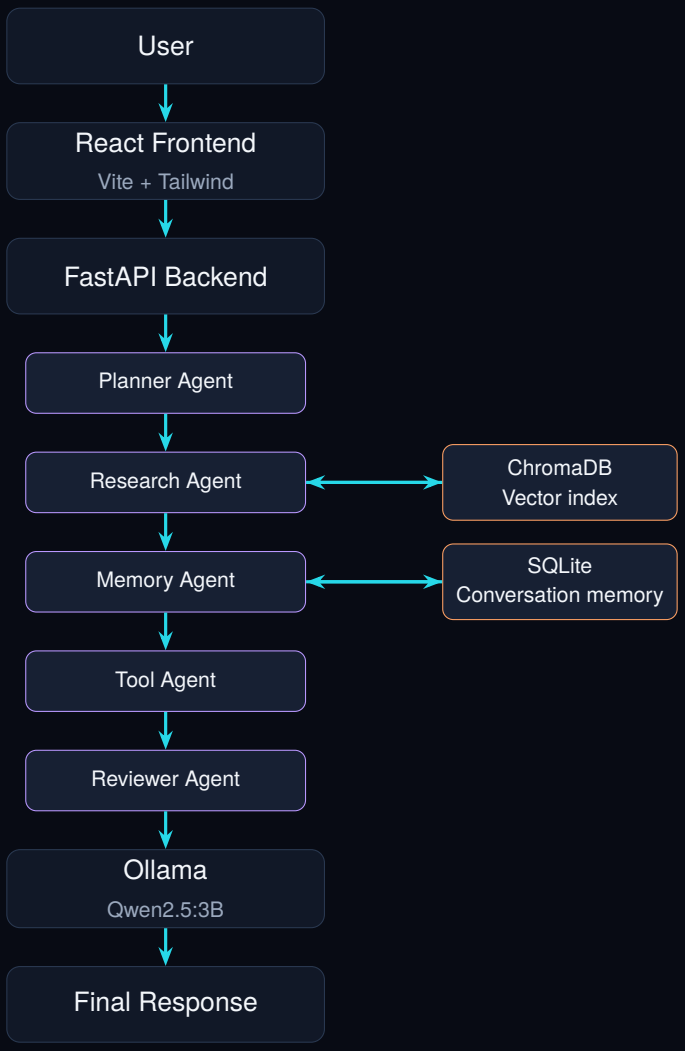
The orchestration layer maintains a shared request state containing the user intent, workspace identifier, retrieved evidence, memory context, tool outputs, and review status. Agents do not directly mutate arbitrary application data. Instead, they return structured messages that the coordinator validates and passes to the next stage. This makes the execution trace suitable for debugging and later analytics.

Guardrail

The Reviewer Agent is not a guarantee of truth. It is a control point that can detect missing evidence, unsupported claims, or incomplete instructions. The product should present uncertainty honestly rather than manufacture confidence.

6 System Architecture

The architecture separates user experience, service orchestration, agent reasoning, local model inference, and persistence. This separation keeps the product understandable while allowing each layer to evolve independently.



6.1 Data Boundaries

PDF binaries and extracted text remain in the local application environment. ChromaDB stores document chunks and their embeddings for semantic retrieval. SQLite stores users' local session records, conversation turns, document metadata, and compact memory entries. Ollama serves the local language model through a local process boundary. No component requires a remote API for the core workflow.

7 Data and Retrieval Pipeline

Document intelligence begins before a question is asked. During ingestion, the backend validates the file type, extracts text, normalizes whitespace, segments content into retrievable chunks, and attaches metadata such as document name, page number, section heading, and workspace identifier. The resulting records are written to ChromaDB with stable identifiers so that evidence can be traced back to the source document.

At query time, the user request is transformed into a retrieval operation. The Research Agent searches the local vector index, selects the most relevant passages, and builds an evidence bundle. The bundle is not simply a list of text fragments. It includes source metadata and ordering information so the reviewer can assess whether the final answer is grounded in an appropriate part of the corpus.

8 Workflow and State Management

The workflow is designed to provide visible progress at every meaningful stage. Users should never have to wonder whether a PDF is still indexing, whether retrieval returned nothing, or whether the reviewer is requesting a revision. Each state can be represented in the dashboard and logged locally for diagnostics.

Step	System action	Visible status
1. Upload	Validate and store the PDF locally	File received
2. Index	Extract, chunk, embed, and write to ChromaDB	Indexing progress
3. Request	Accept a question, summary, or operation	Request received
4. Plan	Convert intent into ordered subtasks	Plan prepared
5. Research	Retrieve relevant context	Evidence collected
6. Memory	Recall scoped prior conversation context	Memory attached
7. Tools	Execute local structured operations	Tool result available
8. Review	Validate grounding and completeness	Response approved
9. Display	Render answer, evidence, and trace	Ready for follow-up

8.1 Failure and Recovery

A robust offline system must handle ordinary local failures: a malformed PDF, an unavailable model, a locked database, an empty retrieval result, or insufficient memory. AgentFlow AI should fail explicitly and recover locally. For example, if no evidence meets the retrieval threshold, the assistant should say that the indexed material does not support a confident answer and offer a narrower query. If Ollama is unavailable, the dashboard should display a model health message rather than presenting an apparently valid but incomplete response.

Operational posture

The system treats observability as part of the product, not as an afterthought. A local trace is useful to developers, but a concise user-facing status is equally important for trust.

9 Technology Stack

The proposed stack is intentionally compact. Each component is widely understood, locally deployable, and replaceable. The goal is to reduce infrastructure friction while retaining enough structure for a credible production path.

Layer	Technology	Role in AgentFlow AI
Frontend	React	Component-based dashboard and interaction state
Build system	Vite	Fast local development and optimized production builds
Styling	Tailwind CSS	Consistent dark visual system and responsive layouts
Backend	Python and FastAPI	API endpoints, orchestration, validation, and streaming
Model runtime	Ollama	Local model serving and lifecycle management
Language model	Qwen2.5:3B	Resource-conscious local reasoning and generation
Vector database	ChromaDB	Local semantic document retrieval
Relational storage	SQLite	Conversations, metadata, and memory records

9.1 Why This Stack

React and Vite support a responsive experience without adding a heavy frontend platform. FastAPI provides typed, testable Python endpoints suited to AI orchestration. Ollama abstracts local model execution and gives the architecture a clear model boundary. ChromaDB is appropriate for a prototype and early deployment because it can operate locally with minimal operational overhead. SQLite is reliable, portable, and sufficient for a single-user or small-team workspace.

The architecture keeps provider-specific calls behind adapters. This means Qwen2.5:3B can be replaced by another compatible local model, and ChromaDB can later be exchanged for a larger on-premise vector service without changing the product concepts exposed to users.

10 AMD Optimization Strategy

AgentFlow AI is designed around local inference, making it naturally aligned with AMD hardware acceleration. Compatible AMD Radeon GPUs can support accelerated model execution through ROCm-enabled workflows, reducing latency and improving the responsiveness of retrieval-heavy conversations. The advantage is particularly relevant when users process multiple documents or run several agent steps in sequence.

A local system does not need to treat hardware as a fixed assumption. The runtime can detect available compute, select a suitable model configuration, and expose a health panel describing the active device. On a CPU-only machine, the product remains functional with smaller workloads. On a compatible AMD GPU, inference can be accelerated while the user retains the same privacy model and interface.

Optimization area	AMD opportunity	Expected product effect
Inference	ROCm-compatible GPU execution	Lower response latency and higher throughput
Concurrent agents	Efficient scheduling of sequential model calls	Smoother orchestration for complex requests
Document workloads	Faster embedding and retrieval preparation	Shorter indexing time for local corpora
User feedback	Runtime health and device visibility	Clearer performance expectations
Deployment choice	CPU fallback plus GPU acceleration	Broader hardware compatibility

10.1 Benchmarking Plan

The evaluation should compare CPU-only and AMD-accelerated configurations using identical documents, prompts, and model settings. Key measurements include time to first token, complete response latency, PDF indexing duration, peak memory usage, and retrieval consistency. The objective is not to claim a universal speedup; it is to demonstrate a transparent, reproducible relationship between local hardware and user experience.

AMD alignment

The strongest story is architectural rather than cosmetic: AgentFlow AI places privacy-sensitive inference at the edge, where AMD compute can make local intelligence practical for real workloads.

11 Security, Privacy, and Governance

Privacy is a system property, not a marketing label. AgentFlow AI minimizes exposure by keeping files, extracted text, embeddings, prompts, responses, and memories inside the local deployment boundary. The default configuration does not require cloud credentials, telemetry services, or remote model calls. This gives administrators a clear starting point for reviewing data flows.

The product should still acknowledge that local does not automatically mean secure. A compromised workstation, weak file permissions, or unmanaged backups can expose sensitive documents. Therefore, the implementation should make storage paths explicit, avoid unnecessary copies, and provide a straightforward way to remove a document and its derived index records.

Control	Implementation direction
Data locality	Store source files, indexes, and conversations on the controlled host
Workspace isolation	Attach workspace identifiers to documents, vectors, and memories
Deletion	Remove source content and derived records through one user action
Auditability	Retain local execution traces without exporting content
Model transparency	Display the active local model and runtime health
Least privilege	Separate application storage from system directories

11.1 Responsible AI Position

The assistant should distinguish retrieved evidence from generated interpretation. It should avoid presenting legal, medical, or compliance decisions as professional advice. Where evidence is incomplete or conflicting, the response should surface uncertainty and point the user to the relevant passages. A Reviewer Agent can support these behaviors, but product copy and interface design must reinforce them.

12 Delivery Roadmap and Future Scope

The project can mature in deliberate stages. The hackathon release should prove the complete local loop rather than attempting every possible modality. A narrow, reliable experience is more persuasive than a broad collection of unfinished features.

Phase	Focus	Outcome
Foundation	PDF upload, extraction, ChromaDB indexing, basic chat	Searchable private document workspace
Orchestration	Planner, Research, Memory, Tool, and Reviewer agents	Observable multi-agent execution
Productization	History, evidence drawer, health status, error recovery	Cohesive investor-quality experience
Acceleration	AMD GPU profiling and ROCm-compatible deployment	Faster local inference path
Expansion	Multi-document reasoning, multimodal input, enterprise controls	Broader deployment value

12.1 Future Scope

- **Voice assistant:** local speech recognition and spoken responses for hands-free review.
- **Multi-document reasoning:** compare policies, contracts, or research findings across a corpus.
- **Multimodal AI:** interpret diagrams, tables, and scanned pages with local vision models.
- **Local code execution:** run controlled analysis tools inside an isolated sandbox.
- **Enterprise deployment:** role-based access, shared workspaces, audit controls, and managed model policies.
- **Adaptive acceleration:** select quantization and runtime settings according to available AMD hardware.

The roadmap preserves the core promise throughout: additional capability should not require surrendering control of private information.

13 Advantages and Evaluation

AgentFlow AI has a differentiated advantage because privacy, offline operation, and agent modularity reinforce one another. A local knowledge base eliminates cloud dependency; the multi-agent design turns complex reasoning into manageable stages; the dashboard makes those stages understandable. Together, these choices form a product rather than a collection of libraries.

Advantage	Why it matters	Proof point
Complete privacy	Sensitive files remain under organizational control	Network-independent end-to-end demo
Fully offline	Useful in restricted or unreliable environments	Model and data services run locally
Fast response	Local retrieval avoids remote round trips	Progressive agent status and benchmark results
Modular agents	Capabilities can be improved independently	Structured orchestration trace
Expandable architecture	New tools and models can be introduced cleanly	Adapter-based service boundaries
Local knowledge base	Institutional context remains available on premises	ChromaDB workspace indexing
No cloud dependency	Reduced vendor, network, and data-transfer risk	No external API required

13.1 Hackathon Demonstration Script

A compelling demonstration begins with a confidential-looking document set and a disconnected network indicator. The presenter uploads a PDF, shows the local indexing state, and asks a question whose answer requires more than keyword matching. The dashboard then reveals the planner, research, memory, and review stages before presenting an answer with source references. A follow-up question demonstrates persistent memory. Finally, the runtime panel shows the local Ollama model and the potential AMD acceleration path.

13.2 Success Measures

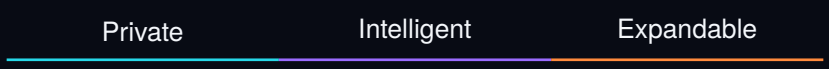
Success is measured by completion of the full workflow, evidence visibility, answer usefulness, recoverable errors, and clarity of the privacy boundary. Technical performance matters, but trust is the central product metric: users should understand where their data went, what the system used, and how to challenge an answer.

14 Conclusion

AgentFlow AI demonstrates a practical direction for private document intelligence: multiple specialized agents collaborating entirely on the user's own machine. The system brings together PDF ingestion, semantic retrieval, conversation memory, local model inference, and a modern dashboard without depending on cloud APIs. Its architecture is intentionally understandable, because trustworthy AI systems need more than impressive outputs; they need boundaries, evidence, and operational clarity.

The project is well suited to the AMD AI DevMaster Hackathon 2026 Track 2 because it connects an urgent product problem with a technically credible local AI strategy. AMD Radeon GPUs and ROCm-compatible workflows can help make offline inference faster and more responsive, while the underlying design remains useful across different hardware configurations.

The near-term objective is a polished end-to-end prototype that proves the private workflow. The longer-term opportunity is an on-premise intelligence platform for organizations whose most valuable documents cannot be casually uploaded elsewhere. AgentFlow AI treats privacy not as a limitation to work around, but as the foundation of a better product.



Private by architecture. Intelligent by design.

References

1. AMD. *ROCm Documentation and Developer Resources*. Available at: https://rocm.docs.amd.com/. Accessed 6 August 2026.
2. Ollama. *Local Model Serving Documentation*. Available at: https://ollama.com/. Accessed 6 August 2026.
3. Chroma. *ChromaDB Documentation: Open-Source Embeddings Database*. Available at: https://docs.trychroma.com/. Accessed 6 August 2026.
4. FastAPI. *FastAPI Web Framework Documentation*. Available at: https://fastapi.tiangolo.com/. Accessed 6 August 2026.
5. React. *React Documentation*. Available at: https://react.dev/. Accessed 6 August 2026.
6. SQLite. *SQLite Documentation*. Available at: https://www.sqlite.org/docs.html. Accessed 6 August 2026.
7. Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, 2020.
8. Qwen Team. "Qwen2.5 Technical Report." *arXiv preprint*, 2024. Available at: https://arxiv.org/. Accessed 6 August 2026.
9. National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework*. Available at: https://www.nist.gov/itl/ai-risk-management-framework. Accessed 6 August 2026.
10. ISO. *ISO 32000-1:2008, Document Management – Portable Document Format – Part 1: PDF 1.7*. International Organization for Standardization.